1) ¿Qué dice el paper de Naur?

Programar (Diseño e implementación de soluciones programadas) no es la producción de texto, los programadores construyen una teoría, esta es el conocimiento que se tiene sobre ciertos aspectos del programa y la capacidad para explicarlos. Alguien que tiene la teoría, no la puede expresar en términos de reglas, y su educación se debe basar en enfatizar el ejercicio de construir una teoría.

La teoría trasciende la documentación ya que, el programador teniendo su teoría puede:

- Explicar como la solución se relaciona con el mundo real, como se mapea
- Explicar Por qué cada parte del programa es como es
- Ser capaz de responder a cualquier demanda de modificación

Es difícil/imposible realizar modificaciones de un programa ya existente si no se fueron adaptando con programadores que tienen su teoría. No es barata la modificación en ese aspecto.

La vida de un programa consiste con el suporte continuo por parte de programadores que tienen su teoría. Y su muerte ocurre cuando se disuelve el equipo que tiene su teoría, se puede revivir con un nuevo equipo que obtenga la teoría trabajando cerca del equipo que si la tiene.

2) ¿Qué dice el paper de Brooks? Detallar las 4 dificultades esenciales

En primer lugar, dice que la construcción del software involucra la complejidad esencial y accidental, siendo la primera la complejidad del dominio del problema, la cual no se disminuir, en cambio la accidental es la que te agrega un lenguaje de programación y el mapeo de estos en un lenguaje maquina en cierto especio y velocidad preterminado.

Esencia irreducible en los sistemas de software modernos:

a- Complejidad: No hay dos partes iguales, hay un incremento de partes diferentes que encima interactúan entre si de una manera no lineal. De esta

complejidad viene la difícil comunicación entre el equipo, fallas en el producto, incremento de costos, retraso del tiempo establecido, etc.

b- Conformidad: Según las necesidades del ser humano, se agrega complejidad en la adaptación del problema para coincidir con las interfaces pedidas.

c- Cambiabilidad: Hay una presión para cambiar, ya que el software en un sistema expresa su función, y la función es la parte que más siente dicha presión para poder extenderse. También ocurre al necesitar que el software se adapte a una nueva máquina distinta a la que fue diseñada.

d- Invisibilidad: El software es algo invisible que no tiene una representación geométrica al no estar por su naturaleza en un espacio. Al intentar diagramarlo aparecerán muchos grafos dirigidos superpuestos aumentando la complejidad. Esto también dificulta la comunicación y la representación mental del software.

Resolver aspectos accidentales: Lenguajes de alto nivel, time-sharing, programación orientada a objetos, programación gráfica, etc.

Propuestas para resolver problemas esenciales:

a- Comprar: Equipar a los trabajadores con programas de dibujos, archivos, spreadsheets, y computadoras personales -> Programas bien documentados.

b- Prototipado rápido: La extracción iterativa y el refinamiento de los requerimientos del producto es fundamental en el desarrollo de software, es por eso que tener herramientas para emular interfaces y funciones importantes es vital.

c- Desarrollo iterativo incremental: Mediante un diseño top-down donde las funciones más complejas van creciendo orgánicamente “grow, not build”.

d- Buenos diseñadores:

    1- Identificar a los mejores diseñadores
    2- Asignarles un mentor
    3- Mantener un plan de desarrollo de carrera con cursos cortos
    4- Interacción entre ellos.

3) ¿Como se vincula no silver bullet con el paper de Naur?

Desarrollo iterativo incremental enfatiza la construcción de la teoría de un programa

4) ¿Qué dificultad esencial resuelve el prototipado y refinamiento continuo?

Conformity, ya que el tener las funciones e interfaces más importantes primero ataca a las necesidades del cliente, las cuales pueden ser ambiguas e inespecíficas.

5) ¿De que habla what is software design?

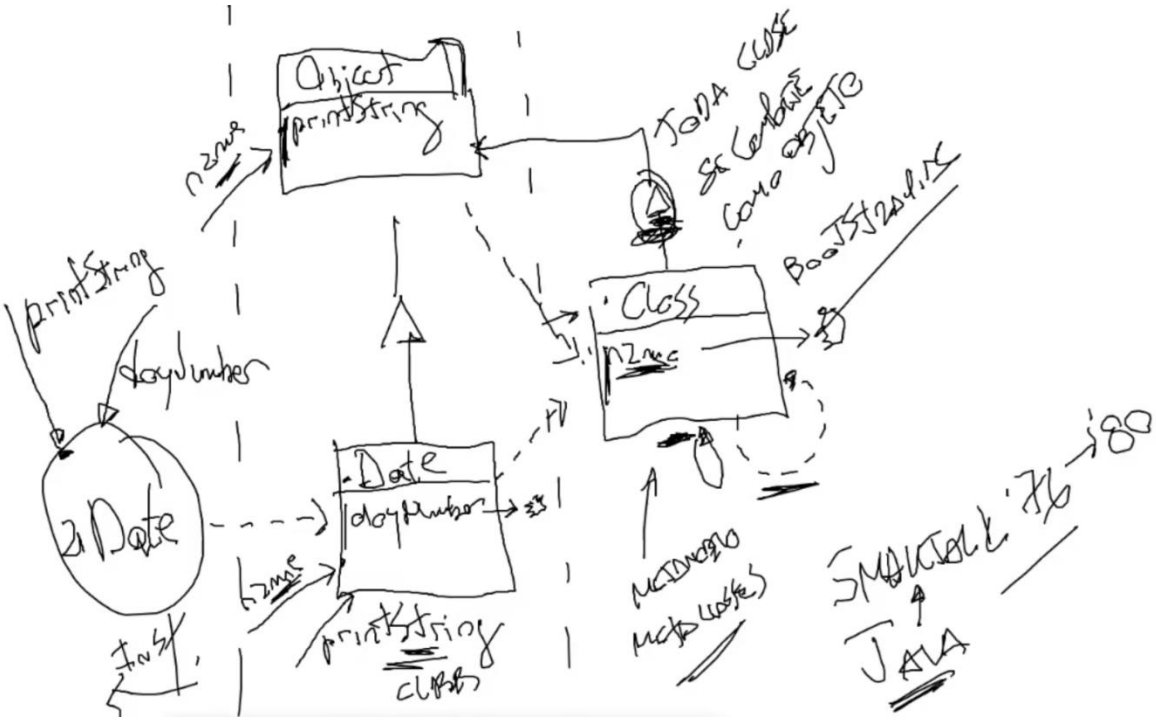
- El código Fuente es el documento que representa el diseño de software
- El software es fácil de construir (Compiladores y linkers)
- El software es caro de diseñar por su complejidad (y porque tiene muchas partes: codear, testear, debugear; Todo es diseño).
- Programar es una actividad de diseño, al igual que particularmente lo son testing y debuggeo. Diseñar software es un ejercicio de manejar complejidad
- Mientras antes se codee, mejor será el diseño porque se ira refinando desde un primer momento
- Métodos formales de validación no son de mucho uso en el mundo real del desarrollo de software
- Notaciones de diseño de software son muy útiles como documentación auxiliar
- El desarrollo de software es más un “craft” que una disciplina de ingeniería por la falta de rigor en el proceso critico de validad y mejorar el diseño
- Lenguajes de programación como C++ ayudan a mejora el diseño de software

6) Relacionar el paper de what is software design con Programming as theory building

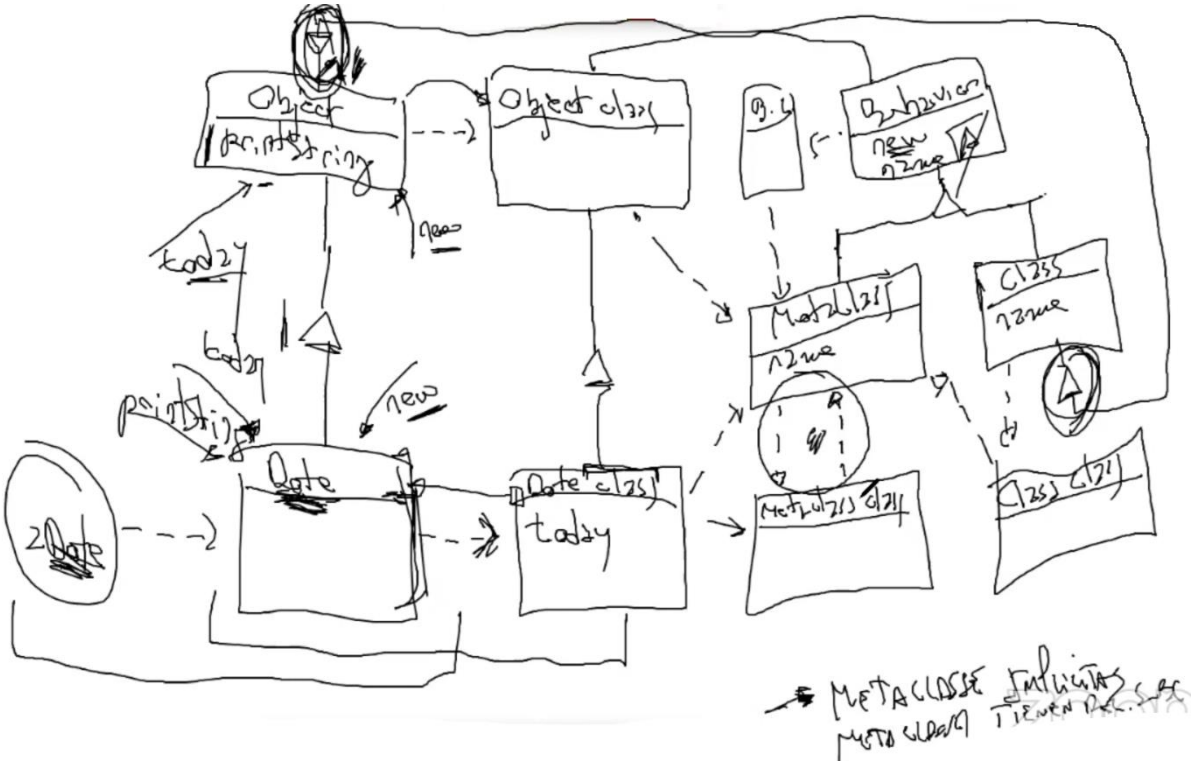
Ambos sostienen que la documentación no es la que todos conocemos, si no que trasciende a algo más, uno habla de una teoría que adquiere el programador, mientras que otro habla del código fuente en sí. Ambos están de acuerdo en que programar no es la producción de texto, si no que es una actividad de diseño la cual es compleja y caro por la complejidad del dominio del problema en sí.

7) Hablar del metamodelo de Smalltalk

El method look-up es: Cada vez que un objeto recibe un mensaje, la implementación de ese mensaje se va a buscar a la clase del objeto. Si no lo encuentro voy a la superclass



Meta clases: Clases que definen el comportamiento de las clases
Limitación del 1er modelo: No podés dar comportamiento especializado a las clases



Smalltalk tiene metaclases implícitas (cuando creas una clase, se crea tu metaclase automáticamente).

Partes importantes:

- Behaviour subclasifica de object: Todo lo que representa comportamiento, se comporta como un objeto
- Object Class subclasifica de class: Todas las clases se comporten como clases
- La circularidad que cierra la regresión al infinito.

8) ¿Qué opina Alan Kay del metamodelo de Smalltalk?

Él opinaba que las metaclases habían probado ser confusas para los usuarios, traían más confusión que algo bueno. A pesar de que esto se hizo para hacer más sencilla la inicialización de instancia, era una mejoría muy menor de lo que ya había hecho Smalltalk-76.

9) ¿Qué técnicas que vimos en la materia atacan dificultades esenciales?

El tDD ayuda a atacar changeability porque te deja más listo para hacer cambios. Los tests te ayudan a asegurarte que no rompiste todo el resto

10) ¿Cuáles fueron las influencias de Smalltalk? (¿Qué tomo de lisp, simula, y sketchpad Alan Kay?)

- Sketchpad: un sistema de comunicación grafico entre el hombre y la máquina. Este fue la invención de los gráficos interactivos modernos de la computadora, las cosas eran descriptas haciendo un dibujo "master" el cual producía "instancias de dibujo", y los controles y dinámicas estaban dados por delimitaciones en una forma gráfica. Fue el primero en tener ventanas que hacían zoom y clipping. Pero Alan Kay había visto esto de master e instance drawing en Simula como actividades y procesos, al igual que en muchos otros lugares. Ahí pensó: El principio básico de los diseños recursivos es hacer que las partes tengan el mismo poder al igual que en conjunto

- Lisp: Le llamo la atención como el lenguaje se suponía que se basaba solo en funciones, pero sus componentes más importantes: Expresiones lambda, quotes (Devuelve un objeto sin evaluarlo), y condicionales; no eran funciones en lo absoluto, si no que se llamaban formas especiales.
  A parte, no había solo EXPRS que evaluaban argumentos, también había FEXPRS que no lo hacían. Alan pensó en por qué no tener solo FEXPRS y forzar la evaluación cuando sea necesario. Ahí pensó: Toma la cosa más dura y profunda que necesites hacer, hazla grande, y luego toda cosa más fácil de ella

- Flex machine: Realizada junto con Ed Cheadle en un lenguaje de programación de alto nivel puesto que quería que no quería que solo fuera para profesionales en computación, por eso eligieron Joss. La belleza de Joss era su extrema atención al diseño para el usuario final

- Doug Engelbart: Influencio a la Flex machine en el hecho de hablar de un aumento del intelecto humano mediante un vehículo interactivo para navegar a través del espacio conceptual.

- Grail: Sucesor grafico de Joss, el cual estaba en la rand tablet que capturaba gestos humanos. Alan Kay se inspiró en su interfaz elegante.

11) Hablar de lo pedagógico en Smalltalk

La charla que escucho Alan Kay seguía las ideas de Piaget, las cuales sostenían que, en términos de situaciones complejas, las escuelas eran un mal lugar para aprender. Alan Kay pensó en como la educación debería ser rediseñada, siendo la computación un sistema de representación con poderosas metáforas para lidiar con complejidades.
También decía que el pensamiento icónico era más fácil que el simbólico y se aprendía antes y que eso justificaba ir más hacia Smalltalk e interfaces gráficas amigables)

12) ¿Qué diría Platón sobre el metamodelo de Smalltalk?

A Platón le parecería una broma pesada, puesto que los objetos actúan como idealización de conceptos (ideas), de las cuales manifestaciones pueden ser creadas. Pero que las ideas son también manifestaciones (de la idea-idea), y la idea-idea es un tipo de manifestación-idea. Al igual que hablar de la class class, etc.

13) ¿Cuáles fueron las críticas de Alan Kay a Smalltalk?

Metaclases, cambiar la fuente de Smalltalk (Antes pusieron una más legible), y hacer a los blocks más como expresiones lambdas (Diferentes tipos de instanciación y de scope fue una mala idea).

14) ¿Qué era lo bueno/malo de lisp según Alan Kay?

lo bueno: El intérprete de Lisp estaba escrito en si mismo, lo que prometía en que todo era una función (Aunque no fue así).

lo malo: Tiene una sintaxis complicada y modelo poco coherente al plantear todo como función, pero utilizar fórmulas especiales como base

15) ¿Cuáles son las formas especiales de lisp que no son funciones? ¿Por qué el cond esta de esta manera?

Expresiones lambda, quotes (Devuelve un objeto sin evaluarlo), y condicionales. El cond decidieron hacerlo así.

Cosas básicas:

1) La colaboración entre objetos es:
   - Sincrónica: Se queda esperando respuesta antes de enviar el siguiente mensaje
   - Dirigida: Para enviar un mensaje, debe conocer a quien
   - Siempre existe un resultado
   - El receptor desconoce al emisor

2) Pasos para sacar código repetido:
   - Mover lo repetido a una nueva abstracción
   - Parametrizar lo que cambia
   - Poner nombre a la nueva abstracción (Importante)
   - Reemplazar el código repetido por el uso de la nueva abstracción

3) Super solo se comporta de manera distinta cuando se le envía un mensaje, y lo que hace es ir a buscar la implementación del mensaje a la super clase de la clase donde está definido el método

4) Heurísticas buenas: Crear objetos completos y válidos. Favorecer el uso de objetos inmutables, no nil, evitar getters y setters, y no modificaciones atómicas